Exp 6: Credit Risk Assessment: Build a credit scoring model to assess the creditworthiness of applicants using historical financial data and perform comparative analysis (Logistic Regression, Random Forest, XGBoost)

Note: Use Evaluation Metrics (Accuracy, Precision, Recall, F1-Score for binary classification), (ROC-AUC, Accuracy, Precision, Recall, Weighted / Macro/ Micro Average F1-Score for Multiclassification task) and Confusion Matrix

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,roc_auc_score,classificat
import pickle
```

```
df=pd.read_csv('/content/drive/MyDrive/credit_risk_assessment_500_samples.csv')
df.head()
```

	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_Amount	Loan_Term_Months	Existing_Loans_Count	De
0	1	59	153267	28	818	196649	12	2	
1	2	49	232745	0	436	175354	60	0	
2	3	35	974945	19	797	662297	48	3	
3	4	63	307164	29	776	831725	48	0	
4	5	28	685626	6	893	498625	60	3	

Next steps: Generate code with df New interactive sheet

```
print(df.shape)
df.info()
df.describe()
```

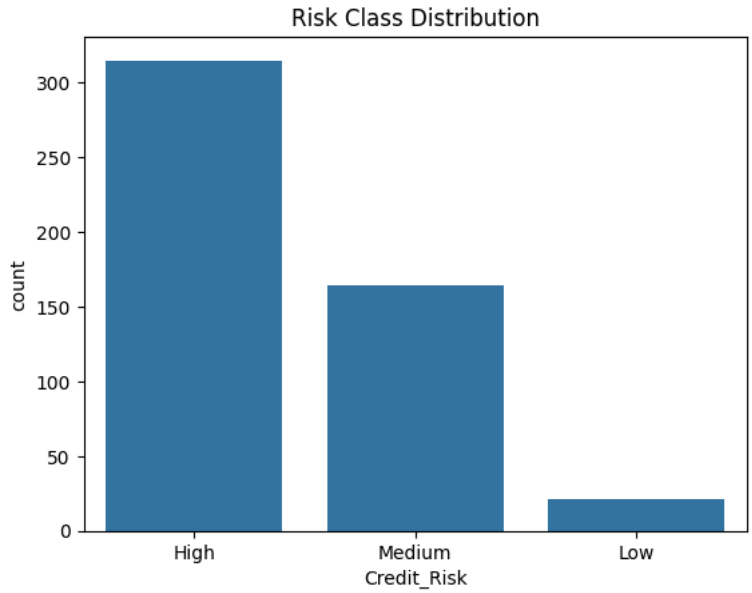
```
(500, 11)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 11 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   Applicant_ID             500 non-null    int64
 1   Age                      500 non-null    int64
 2   Annual_Income            500 non-null    int64
 3   Employment_Years         500 non-null    int64
 4   Credit_Score             500 non-null    int64
 5   Loan_Amount              500 non-null    int64
 6   Loan_Term_Months         500 non-null    int64
 7   Existing_Loans_Count     500 non-null    int64
 8   Debt_to_Income_Ratio     500 non-null    float64
 9   Late_Payments_Last_2Yrs  500 non-null    int64
 10  Credit_Risk              500 non-null    object
dtypes: float64(1), int64(9), object(1)
memory usage: 43.1+ KB
```

1 to 8 of 8 entries Filter

index	Applicant_ID	Age	Annual_Income	Employment_Years	Credit_Score	Loan_Amount	Loan_Term_M
count	500.0	500.0	500.0	500.0	500.0	500.0	
mean	250.5	43.116	827131.654	14.702	596.394	521385.53	
std	144.4818327679989	12.733216939456186	384829.8617613202	8.713497925183917	171.33979518731104	281799.5141543726	17.117037182
min	1.0	21.0	150404.0	0.0	300.0	55901.0	
25%	125.75	32.0	506262.25	7.0	452.0	266621.75	
50%	250.5	44.0	833427.5	15.5	602.0	533640.0	
75%	375.25	53.0	1148353.0	22.0	732.25	765362.5	
max	500.0	64.0	1498464.0	29.0	898.0	999621.0	

Show 100 per page
Like what you see? Visit the data table notebook to learn more about interactive tables.

```
sns.countplot(x="Credit_Risk",data=df)
plt.title("Risk Class Distribution")
plt.show()
```




```
le=LabelEncoder()
df["Credit_Risk"]=le.fit_transform(df["Credit_Risk"])
```

```
df=df.drop("Applicant_ID",axis=1)
```

```
X=df.drop("Credit_Risk",axis=1)
y=df["Credit_Risk"]
```

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=0)
```

```
models={
    "Logistic Regression":LogisticRegression(max_iter=1000),
    "Random Forest":RandomForestClassifier(),
    "XGBoost":XGBClassifier(eval_metric="mlogloss")
    }
```

```
results=[]
for name,model in models.items():
    model.fit(X_train,y_train)
    pred=model.predict(X_test)
    acc=accuracy_score(y_test,pred)
    pre=precision_score(y_test,pred,average="weighted")
    rec=recall_score(y_test,pred,average="weighted")
    f1w=f1_score(y_test,pred,average="weighted")
    f1m=f1_score(y_test,pred,average="macro")
    f1mi=f1_score(y_test,pred,average="micro")
    results.append([name,acc,pre,rec,f1w,f1m,f1mi])
results_df=pd.DataFrame(results,columns=[
"Model","Accuracy","Precision","Recall",
"F1_weighted","F1_macro","F1_micro"])
results_df
```

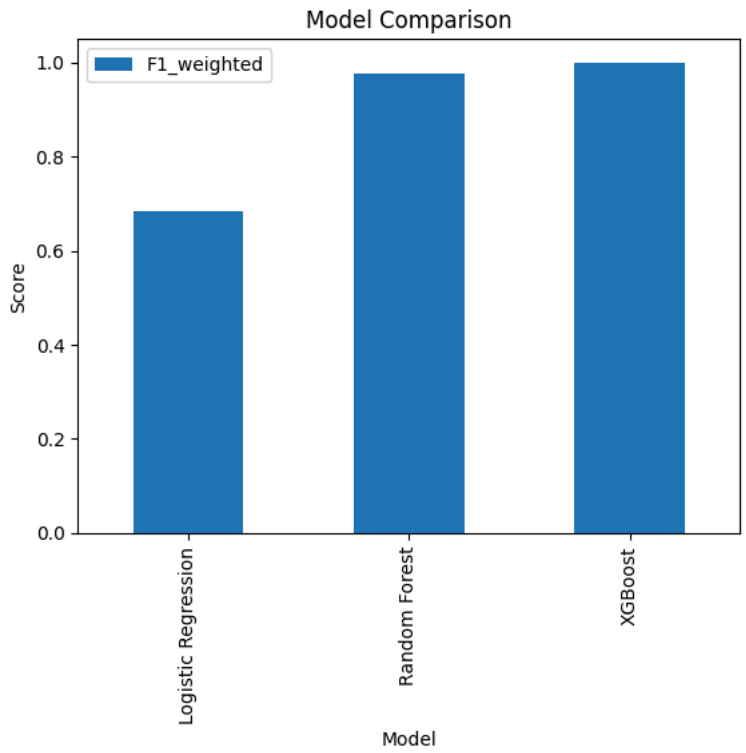
```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

	Model	Accuracy	Precision	Recall	F1_weighted	F1_macro	F1_micro
0	Logistic Regression	0.68	0.699529	0.68	0.683579	0.431938	0.68
1	Random Forest	0.99	0.990370	0.99	0.989380	0.946092	0.99
2	XGBoost	1.00	1.000000	1.00	1.000000	1.000000	1.00

Next steps: Generate code with results_df New interactive sheet

```
results_df.plot(x="Model",y="F1_weighted",kind="bar")
plt.title("Model Comparison")
```

```
plt.ylabel("Score")
plt.show()
```

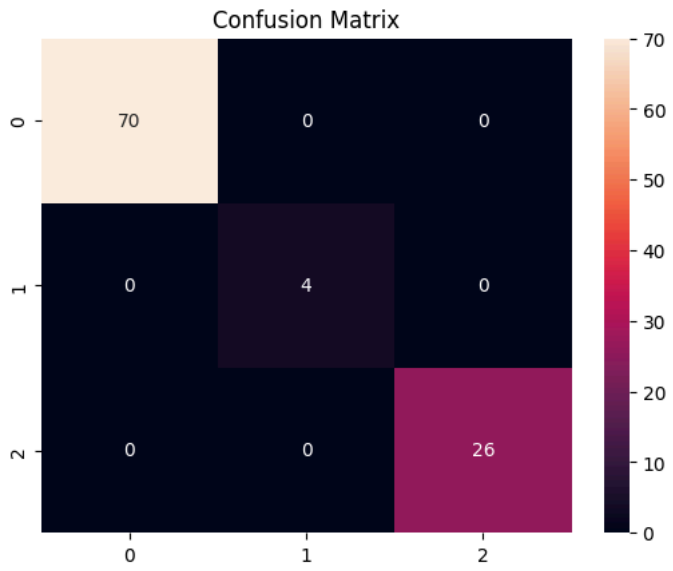



```
best_name=results_df.sort_values("F1_weighted",ascending=False).iloc[0,0]
best_model=models[best_name]
print("Best Model:",best_name)
```

```
Best Model: XGBoost
```

```
with open("best_model.pkl","wb") as f:
    pickle.dump(best_model,f)
```

```
pred=best_model.predict(X_test)
cm=confusion_matrix(y_test,pred)
sns.heatmap(cm,annot=True,fmt='d')
plt.title("Confusion Matrix")
plt.show()
```




```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        70
           1       1.00      1.00      1.00         4
           2       1.00      1.00      1.00        26
```

```
    accuracy                           1.00       100
   macro avg       1.00      1.00      1.00       100
weighted avg       1.00      1.00      1.00       100
```